

HighLoad Invest Ecosystem

Программно-аппаратная учебная экосистема имитации биржевых торгов: от Linux kernel module до мобильного терминала, observability и нагрузочного стенда.

Команда: HighLoad Invest
Преподаватель: Ключев А. О.



Разработка как коллективная инженерная работа

| Ильин Никита Сергеевич

| Алхимовици Арсений

| Яснов Михаил Андреевич

| Плешивцев Кирилл Михайлович

| Соколов Александр Павлович

| Ильин Александр Алексеевич

| Вердиев Фада Ильхам оглы

| Киселев Михаил Васильевич

В этом проекте команда отвечает не только за набор сервисов, а за связность общей модели: требования, архитектуру, реализацию, измеримость и честное описание ограничений.

Учебная брокерская система с полным контуром данных

Источник данных

Linux kernel module создаёт /dev/quotes; Go ingestion читает устройство или включает fallback-генератор.

Backend

API Gateway отдаёт REST/WebSocket API. Core Banking выполняет операции BUY/SELL в транзакции PostgreSQL.

Хранилища

ClickHouse хранит котировки и свечи. PostgreSQL хранит пользователей, счета, сделки и портфель.

Клиенты и проверка

Есть Kotlin/Compose и React Native клиенты, load-tester, Jaeger, Prometheus и Grafana.

Не список библиотек, а работающие сценарии и проверяемые свойства

- GET /api/quotes и GET /api/quotes/{ticker}/candles читают данные из ClickHouse.
- POST /api/trades меняет баланс, портфель и журнал сделок атомарно.
- WS /ws/quotes получает события из Redis Pub/Sub.
- Jaeger показывает путь запроса, Grafana показывает метрики backend.

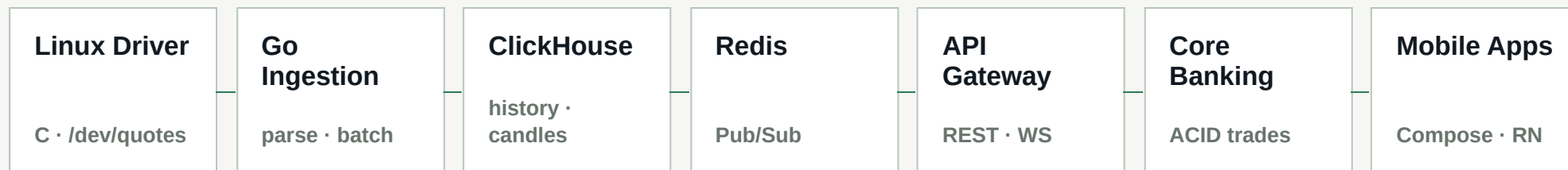
Котировки можно временно потерять в WebSocket, но история остаётся в ClickHouse.

Финансовую операцию нельзя применить частично: PostgreSQL transaction + row locks.

Нагрузка задаётся параметрами load-tester, а не словами «держит много».

Слабые места названы отдельно, без попытки выдать MVP за production-финтех.

Полный путь данных от ядра ОС до пользователя



Market Data

Котировки являются потоком событий и историческим временным рядом.

Trading

Сделки меняют внутреннее финансовое состояние и требуют транзакций.

Observability

Метрики и трассы превращают поведение системы в измеряемый объект.

Функциональные требования

FR-01	Драйвер на C создаёт поток котировок через /dev/quotes .
FR-03	Go ingestion пишет котировки в ClickHouse батчами.
FR-07	API Gateway рассылает realtime-котировки через WS /ws/quotes .
FR-11	BUY списывает деньги и увеличивает позицию в одной транзакции.
FR-12	SELL проверяет доступные лоты и фиксирует сделку в PostgreSQL.
FR-15/16	Android Native и React Native реализуют одинаковый пользовательский сценарий.

Проверяемость

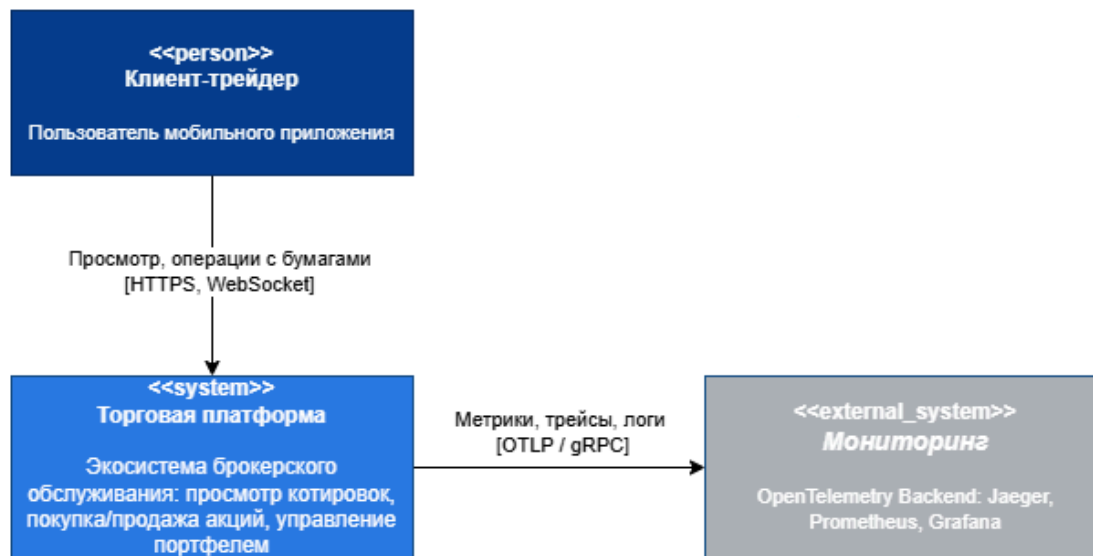
- Для каждого FR в отчёте указан критерий приёмки.
- API проверяется REST/WebSocket-запросами.
- Финансовые требования проверяются транзакционными тестами.
- Мобильные требования проверяются ручным сценарием на двух клиентах.

Полный список: FR-01..FR-19 в отчёте.

Нефункциональные требования

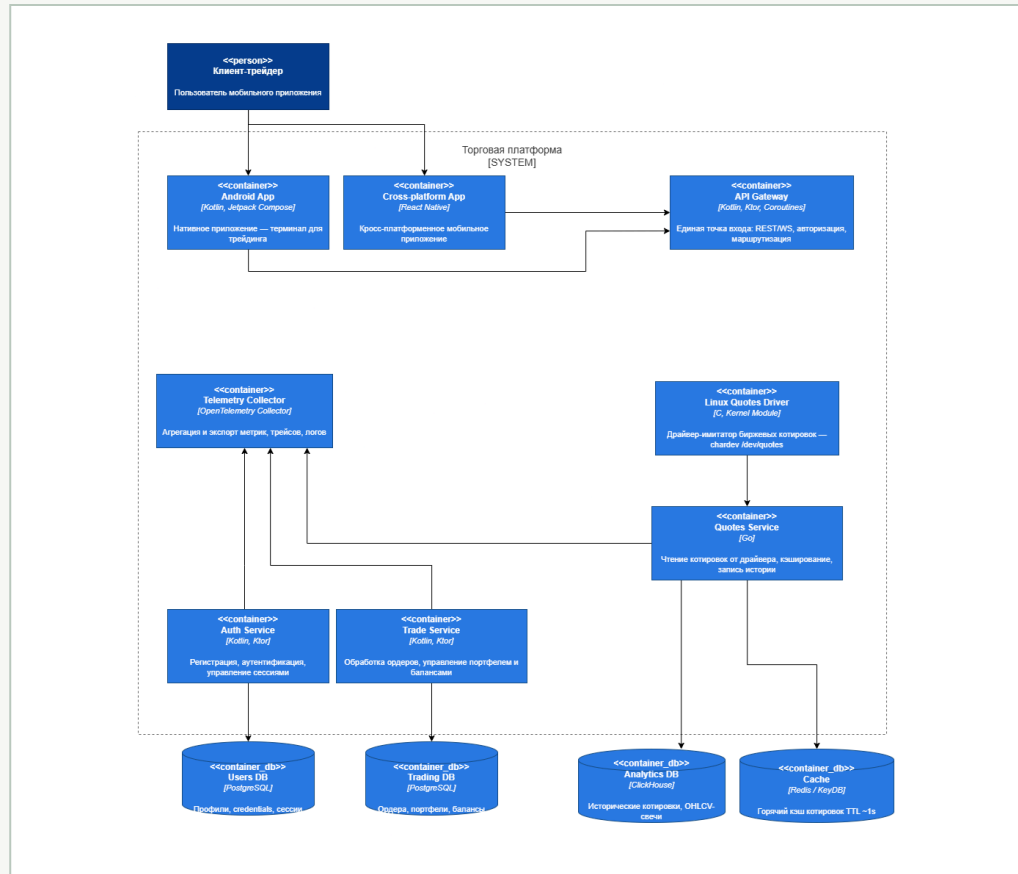
ID	ТРЕБОВАНИЕ	КАК ПРОВЕРЯЕТСЯ
NFR-01	Latency realtime-котировок менее 1 секунды на демонстрационном стенде.	timestamp, WebSocket, Grafana/Jaeger.
NFR-02	Архитектурная готовность к 10 000 активных сессий.	Load Tester, active sessions, p95/p99 latency.
NFR-03	ACID-консистентность финансовых данных.	PostgreSQL transactions, SELECT . . . FOR UPDATE.
NFR-05	Наблюдаемость через метрики и distributed tracing.	OpenTelemetry, Prometheus, Grafana, Jaeger.
NFR-08	Разделение уровней абстракции.	C4, bounded contexts, отсутствие смешения моделей.

Контекст системы

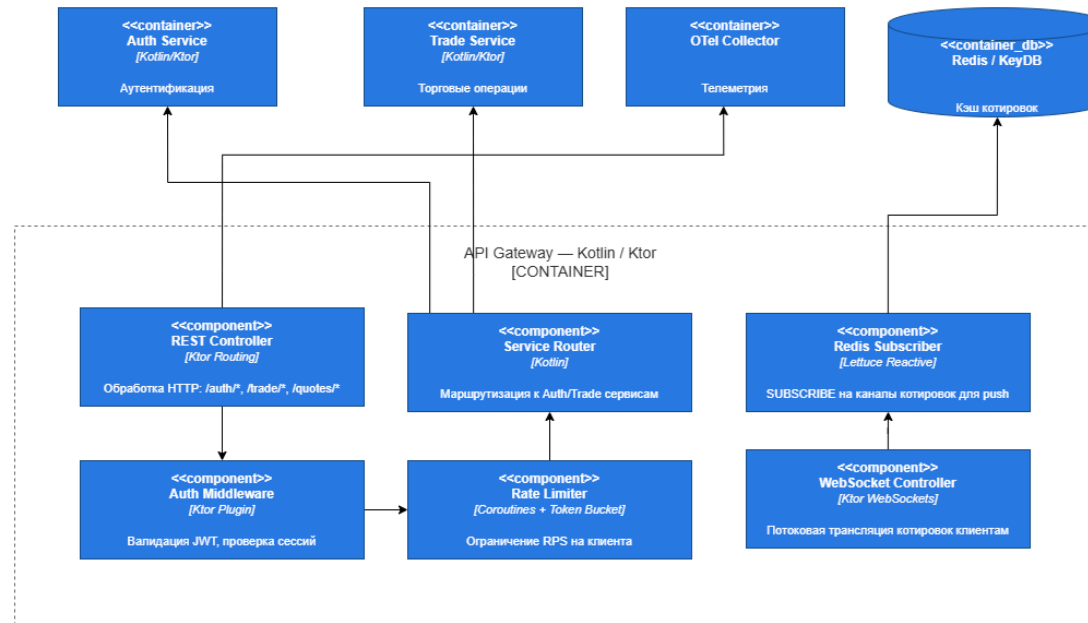


Пользователь, мобильные клиенты, торговая платформа и контур наблюдаемости.

Контейнеры и границы ответственности

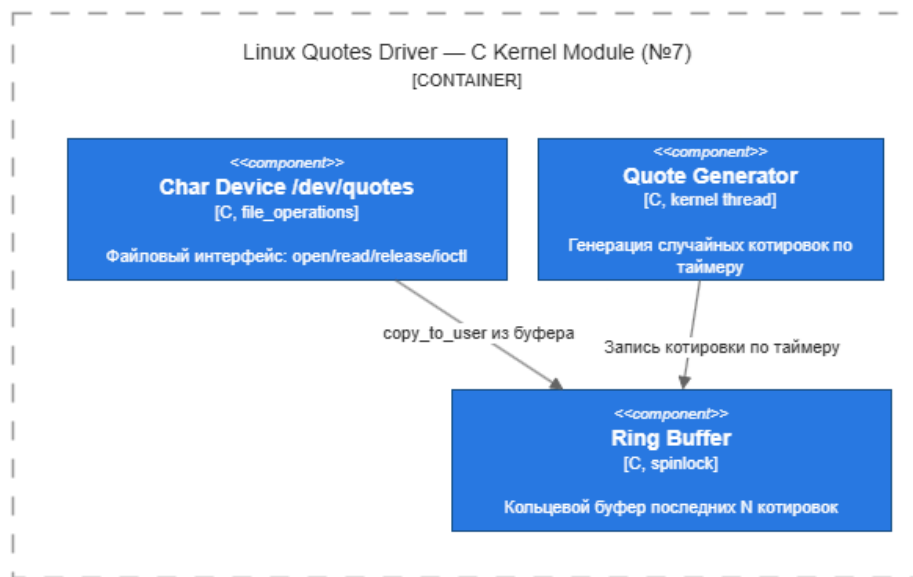


API Gateway



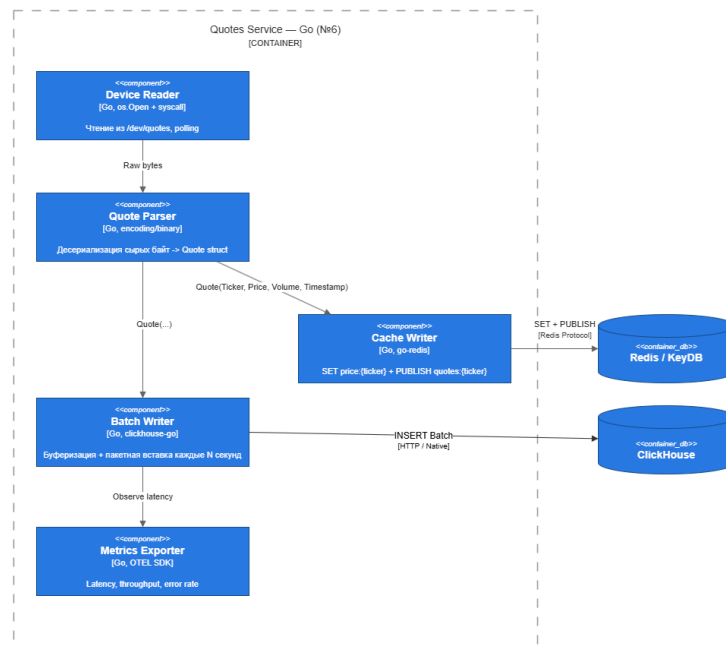
REST Controller, Service Router, WebSocket Controller и Redis Subscriber.

Linux Quotes Driver



Char Device /dev/quotes, генератор котировок и кольцевой буфер в kernel space.

Quotes Service



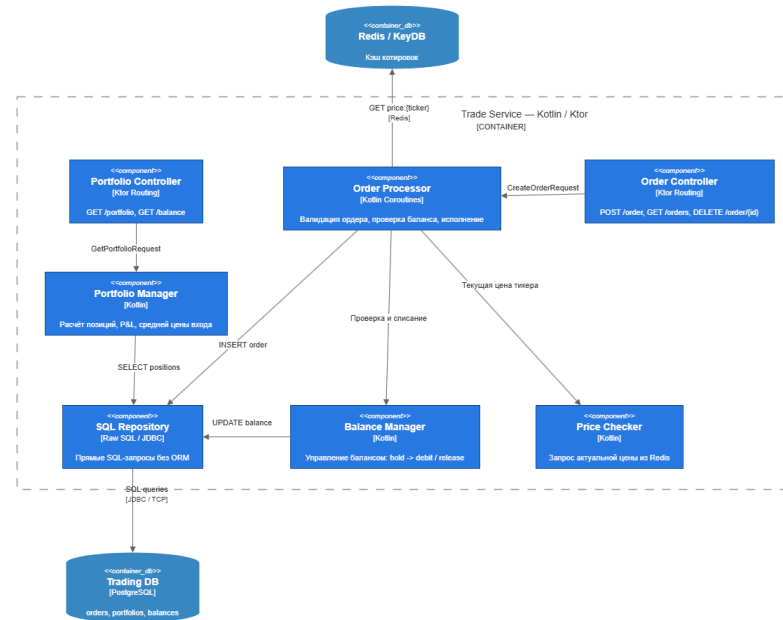
Device Reader, Quote Parser, Batch Writer, Redis publisher и Metrics Exporter.

Auth Service



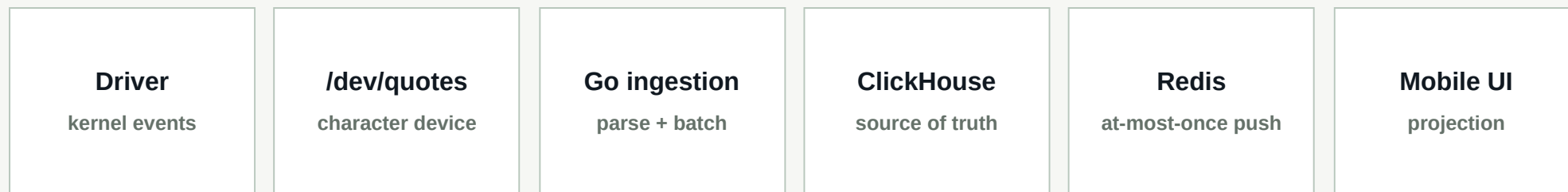
Регистрация, профили пользователей, токены, сессии и PostgreSQL repositories.

Trade Service



Order Processor, Balance Manager, Portfolio Manager, Price Checker и SQL Repository.

Как котировка попадает на экран



ClickHouse

Хранит таблицу quotes; из неё строятся последние цены и OHLCV-свечи.

Redis

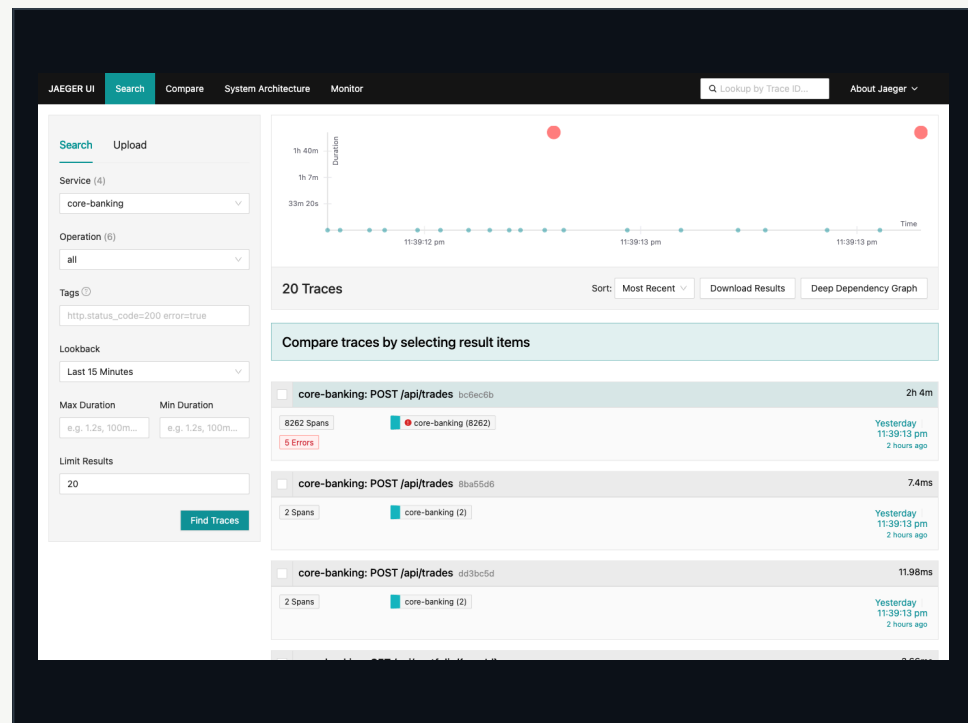
Используется как Pub/Sub-канал quotes:updates, не как источник истины.

Компромисс

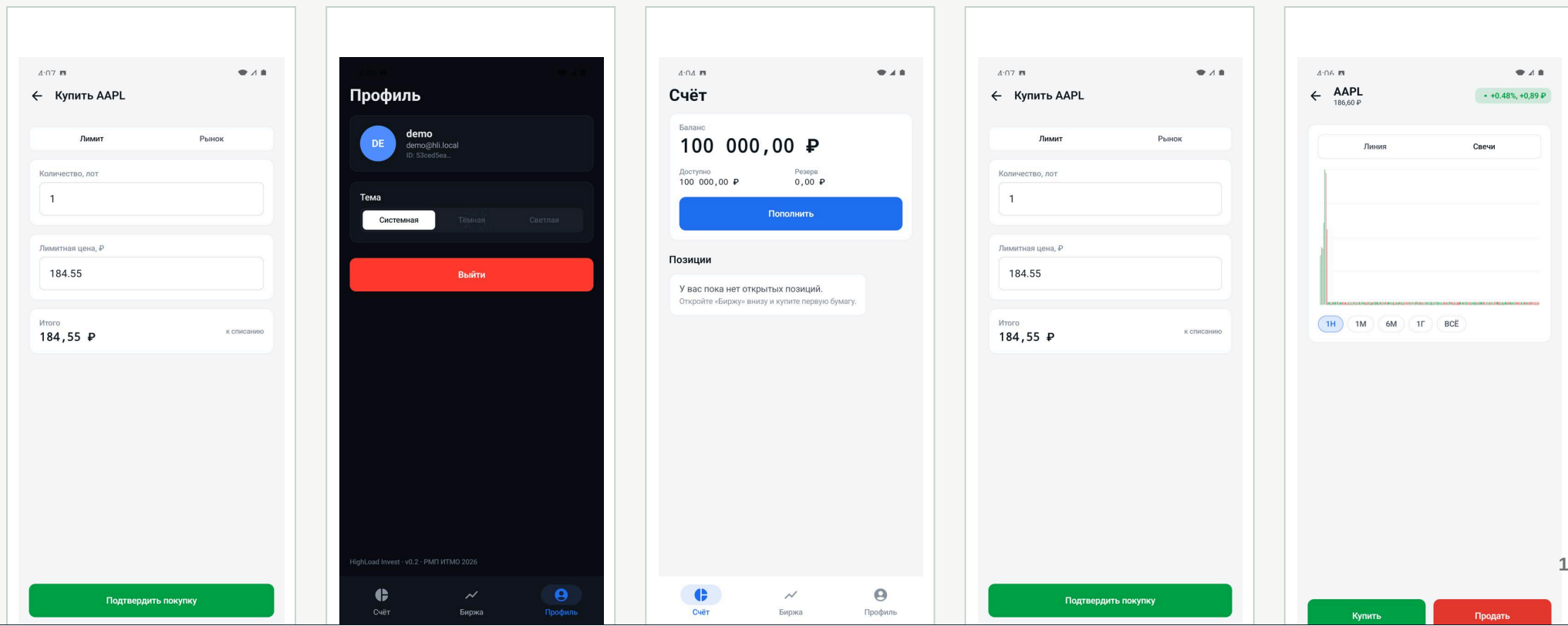
Batch снижает стоимость записи в ClickHouse, но добавляет задержку flush.

Core Banking: что происходит при сделке

- Вход: `userId`, `ticker`, `action`, `lots`, `pricePerLot`.
- `SELECT ... FOR UPDATE` блокирует счёт пользователя на время операции.
- `BUY` проверяет баланс; `SELL` проверяет доступные лоты.
- После проверки обновляются `accounts`, `portfolio`, `trades` и выполняется `COMMIT`.



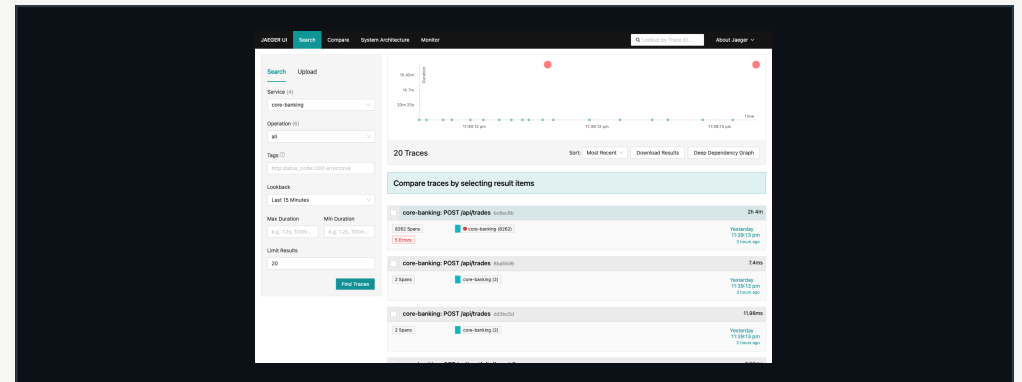
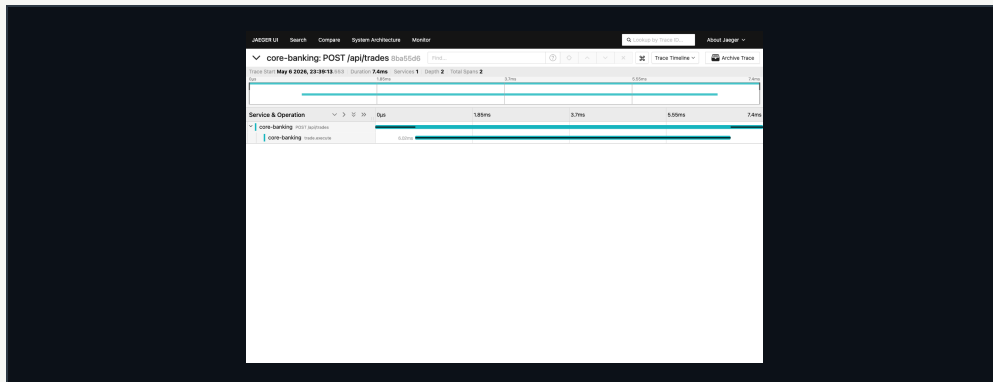
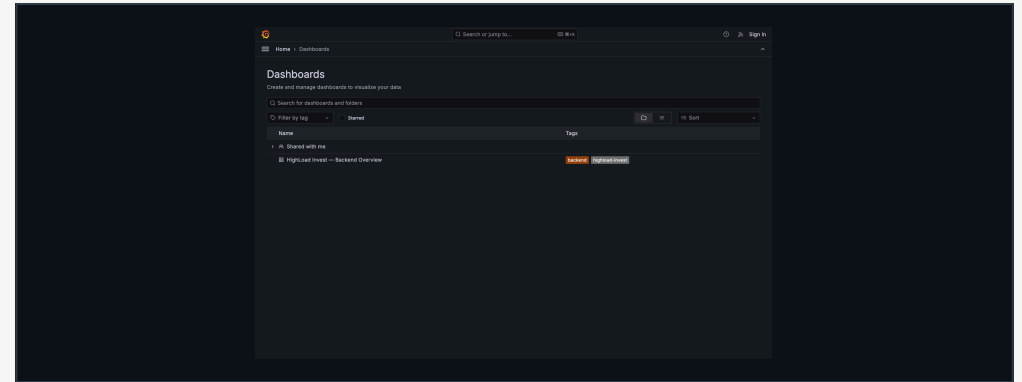
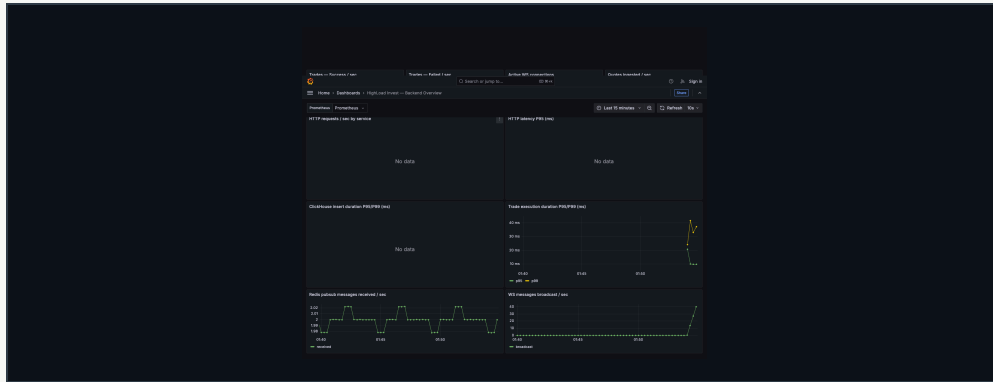
Один пользовательский сценарий в двух реализациях



Native и React Native: что сравниваем

КРИТЕРИЙ	ANDROID NATIVE	REACT NATIVE
Язык и UI	Kotlin + Jetpack Compose	TypeScript + React Native
Состояние	ViewModel + StateFlow	React hooks
Сильная сторона	Контроль lifecycle и Android API	Быстрый MVP и потенциальная мультиплатформенность
Риск	Больше платформенного кода	Runtime-ошибки API без общей схемы DTO
Итог	Оба клиента работают с одним API; различается скорость разработки, контроль платформы и сложность поддержки UI.	

Что видно в Grafana и Jaeger



Как устроен load-tester

Профиль

Боты создают пользователей, читают котировки, выполняют BUY/SELL и держат WebSocket.

Параметры

BOT_COUNT, ACTIVE_REQUESTS, WS_PERCENT, DURATION_SEC, задержки между запросами.

Метрики

RPS, error rate, avg/p95/p99 latency, число WebSocket-сессий, состояние БД и сервисов.

Главная метрика не «запустилось», а latency и error rate при заданном профиле нагрузки.

Где архитектура упрощена и что надо усиливать

- Double для денег — учебное упрощение; нужен decimal domain.
- Цена сделки приходит от клиента; нужен серверный price snapshot.
- Redis Pub/Sub — at-most-once, не подходит для финансовых событий.
- WebSocket broadcast требует backpressure и очередей на сессию.
- DTO дублируются между backend и мобильными клиентами.
- Нужны OpenAPI-контракт и contract tests.
- Load testing должен фиксировать p95/p99, а не только среднее.
- Observability требует SLO, alert rules и trace-id в логах.

Что готово к демонстрации

Стенд поднимается через Docker Compose, клиенты работают с единым API, котировки идут через ClickHouse/Redis/WebSocket, сделки фиксируются в PostgreSQL, а поведение системы видно в Jaeger и Grafana.

- | C4-модель
- | SRS: FR-01..FR-24, NFR-01..NFR-16
- | Два мобильных клиента
- | Jaeger/Grafana evidence
- | Фундаментальный системный анализ